1. LIVE EXAM FOUNDATION (FALL 2026 LIVE PAPER) & OUTPUT TRACING ENGINE (Q2 — 40-50%)

Official Paper Breakdown · Q1 (50%) + Q2 (50%)

Q1: UNIFIED 6-PART 1D ARRAY C++ PROGRAM (LIVE EXAM BLUEPRINT) — 50% WEIGHT

```cpp
#include <iostream>
#include <fstream>
using namespace std;
int main() {
  // a) Input 10 doubles from keyboard:
  double A[10];
  for (int i = 0; i < 10; i++) cin >> A[i];

  // b) Minimum element & print: (CRITICAL: Init min=A[0], NEVER 0!)
  double minVal = A[0]; int minIdx = 0;
  for (int i = 1; i < 10; i++) {
    if (A[i] < minVal) { minVal = A[i]; minIdx = i; }
  }
  cout << "Min: " << minVal << "\n";

  // c) Range check [-7, 7]; else terminate immediately:
  bool inRange = true;
  for (int i = 0; i < 10; i++) {
    if (A[i] < -7.0 || A[i] > 7.0) { inRange = false; break; }
  }
  if (inRange) cout << "elements of A are in range\n";
  else return 0; // Immediate exit

  // d) Repeatedly input i1, i2 until valid (0..9 and i1 != i2) [De Morgan]:
  int i1, i2;
  do {
    cin >> i1 >> i2;
  } while (i1 < 0 || i1 > 9 || i2 < 0 || i2 > 9 || i1 == i2);

  // e) Swap elements i1 and i2 of A using temp variable:
  double temp = A[i1]; A[i1] = A[i2]; A[i2] = temp;

  // f) Save elements to "A.txt":
  ofstream fout("A.txt");
  if (!fout) return 1;
  for (int i = 0; i < 10; i++) fout << A[i] << "\n";
  fout.close();
  return 0;
}
```

Q2: SIGNATURE OUTPUT TRACING ENGINE & THE 8 GOLDEN TRAPS — DR. GORDON TRAPS

- **1. Truncating Integer Div:** `1/(0+3) = 1/3 = 0` (NOT 0.33!). `7/2 = 3`. Float div: `7.0/2 = 3.5`.
- **2. IEEE-754 Div by Zero:** `7.0/(1-1) = 7.0/0.0 = inf` → **Does NOT crash!** Float div by zero yields `inf`.
- **3. Logical NOT `!i`:** `!0 = true (1)`; all non-zero numbers are true, so `!1 = !2 = false (0)`. `else if (!i)` executes ONLY at iteration *i=0*!
- **4. Array Left-Shift:** `B[i]=B[i+1]; i++;` sequentially propagates neighbor into current slot.
- **5. Uninitialized Garbage (G):** `int x;` holds undefined RAM garbage. `x += B[i];` stays Garbage! Write **"Garbage"** or **"G"** in booklet!
- **6. Loop Counter Retention & Bounds:** After `for(i=0;i<5;i++)`, *i* is **5**! `cout<<i` prints **5**. In size-4 array, `A[4]` is out-of-bounds → **Garbage (G)**!
- **7. Pre-Break Trap:** Statements before `if(cond) break;` execute even on the terminating pass!
- **8. Modulo `%` Rules:** `20 % 7 = 6`. Sign follows LHS: `-20 % 7 = -6`; `20 % -7 = +6`.

F26 Q2(a) Trace Box:
```
i=0: 1/3 = 0
i=1: 7.0/0.0 = inf
i=2: 7.0/1.0 = 7
i=3: 7.0/2.0 = 3.5
i=4: x++ = 4.5
post: i = 5
```

F26 Q2(b) Trace Box:
```
B={7,5,3,2,1,1}
i exits at 5
a = 20 % 7 = 6
x is uninit = G
3 2 5 6 G
(or 3 2 5 6 18)
```

F23 Midterm Keys:
```
Q2a: 1 8
     2 14
     3 19
     29 19 26
Q2b: -9 / -9 / 7 / 7
/ G
```

2. ALL SIMILAR EXAM PROBLEMS & ALGORITHMS ACROSS PAST GORDON MIDTERMS & QUIZZES

Fall 2023 · Fall 2022 · Quiz 1 & 2 · Reviews 1 & 4

1. VALIDATED SIZE N & 2 ARRAYS — F22 Q1A / REV 4

```cpp
int n; cin >> n;
while (n < 1 || n > 100) {
  cout << "Re-enter n (1-100): "; cin >> n;
}
double A[100], B[100];
for (int i = 0; i < n; i++) cin >> A[i];
for (int i = 0; i < n; i++) cin >> B[i];
```

2. CYCLIC FORWARD SHIFT — F22 Q1D / REV 4

```cpp
// Shift forward: A[n-1] to 0, A[i] to i+1
double A_sh[100];
A_sh[0] = A[n - 1]; // Wrap last element
for (int i = 0; i < n - 1; i++) A_sh[i + 1] = A[i];
// In-place alternative (backward loop):
double last = A[n - 1];
for (int i = n - 1; i > 0; i--) A[i] = A[i - 1];
A[0] = last;
```

3. ARRAY MERGE / CONCATENATE — F22 Q1D

```cpp
// Merge A and B (size n) into C (size 2n):
double C[200];
for (int i = 0; i < n; i++) {
  C[i]     = A[i]; // 1st half: 0..n-1
  C[i + n] = B[i]; // 2nd half: n..2n-1
}
```

4. DOT PRODUCT & RANGE COUNT — F22 Q1B,C / Q2

```cpp
// Vector dot product: sum(A[i]*B[i]):
double dot = 0.0;
for (int i = 0; i < n; i++) dot += A[i] * B[i];

// Count elements strictly in (0, PI):
const double PI = 4.0 * atan(1.0);
int count = 0;
for (int i = 0; i < n; i++) {
  if (B[i] > 0.0 && B[i] < PI) count++;
}
```

5. IN-PLACE ARRAY REVERSAL — F23 Q1C

```cpp
// Swap i with n-1-i up to n/2 (Zero extra RAM):
for (int i = 0; i < n / 2; i++) {
  double temp = A[i];
  A[i] = A[n - 1 - i];
  A[n - 1 - i] = temp;
}
```

6. ARRAY EQUALITY TEST — MIDTERM REV 1

```cpp
bool equal = true;
for (int i = 0; i < n; i++) {
  // Doubles: if (fabs(A[i]-B[i]) > 1e-7)
  if (A[i] != B[i]) { equal = false; break; }
  // TRAP: NEVER add 'else equal=true;'!
}
if (equal) cout << "Equal"; else cout << "Not equal";
```

7. MENU LOOP ALL-OPTION FLAG — QUIZ 2 / REV 1

```cpp
bool used[4] = { false }; int ch;
do {
  cout << "1=Avg, 2=Min, 3=Dot: "; cin >> ch;
  if (ch == 1) { /* avg */ used[1] = true; }
  else if (ch == 2) { /* min */ used[2] = true;
}
  else if (ch == 3) { /* dot */ used[3] = true;
}
} while (!(used[1] && used[2] && used[3]));
```

8. MEAN, VARIANCE & STDDEV — REV 4 Q1

```cpp
double sum = 0;
for (int i = 0; i < N; i++) sum += X[i];
double mean = sum / N, varSum = 0;
for (int i = 0; i < N; i++) varSum += pow(X[i] - mean, 2);
double variance = varSum / (N - 1);
double stdDev = sqrt(variance);
```

9. FREQUENCY & SERIES & NEW[] — Q2 / REV 4

```cpp
// Frequency count (val 1..7):
int freq[8] = { 0 }, v;
while (cin >> v && v >= 1 && v <= 7) freq[v]++;

// Series loop with tolerance eps=1e-9:
double term = 1.0; int k = 1;
while (1) {
  term *= x / k;
  if (fabs(term) < 1.0e-9) break;
  k++;
}

// Dynamic array: delete[] prevents leak:
double *p = new double[n]; delete[] p; p = nullptr;
```

3. ESSENTIAL LIBRARIES, NUMERICAL CONSTANTS & EXAM ROOM SURVIVAL PROTOCOL

Cheat Codes & Quick References

DIRECTIVES & HEADERS — BOILERPLATE

`#include <iostream>` (cin, cout)
`#include <cmath>` (sqrt, pow, fabs, sin)
`#include <fstream>` (ifstream, ofstream)
`#include <iomanip>` (fixed, setprecision)
`#include <cstdlib>` (exit(0), abs)

MATH & FORMATTING — FORMULAS

Concordia PI Constant:
`const double PI = 4.0 * atan(1.0);`
Float Equality (NEVER ==):
`if (fabs(a - b) < 1.0e-7) { ... }`
Decimal Output (2 places):

OPERATOR PRECEDENCE — ORDER

1. `() [] x++ x--`
2. `++x --x ! (type) sizeof` (R-to-L)
3. `* / %` 4. `+ -`
5. `<< >>` 6. `< <= > >=`

75-MIN TIME PROTOCOL — STRATEGY

00-05m: Scan weights & uninit vars.
05-35m: Solve Q2 tracing tables first! Draw columns, **put boxes around answers**.
35-70m: Write Q1 C++ program cleanly. Init min=A[0], use temp for swap, close files.